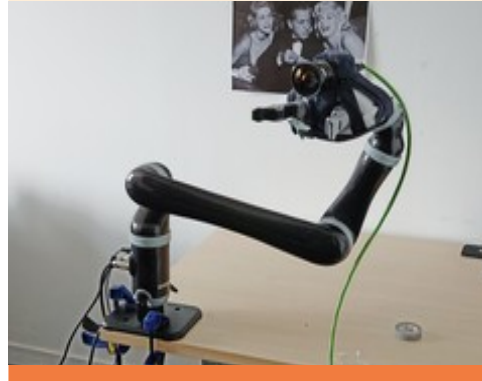


Localization

VA56 - Introduction to Mobile robotics



UTBM - FISE INFO - VA56
Jocelyn BUISSON

Localization

What is localization?

- Localization is the process by which a robot determines its pose (position and orientation) within the environment
- Dead-reckoning
 - Control input integration, open-loop*
- Relative / Absolute
 - Pose estimate between time steps vs reference to a fixed global frame*
- Local / Global
 - Start from an initial guess vs search initial position in reference (map)*
- Map-based / mapless
 - Prior map (e.g., grid, landmarks, point cloud) vs sensor-to-sensor matching*
- SLAM / Pure localization
 - Joint estimate of pose & map vs pose estimation on static map*

Localization

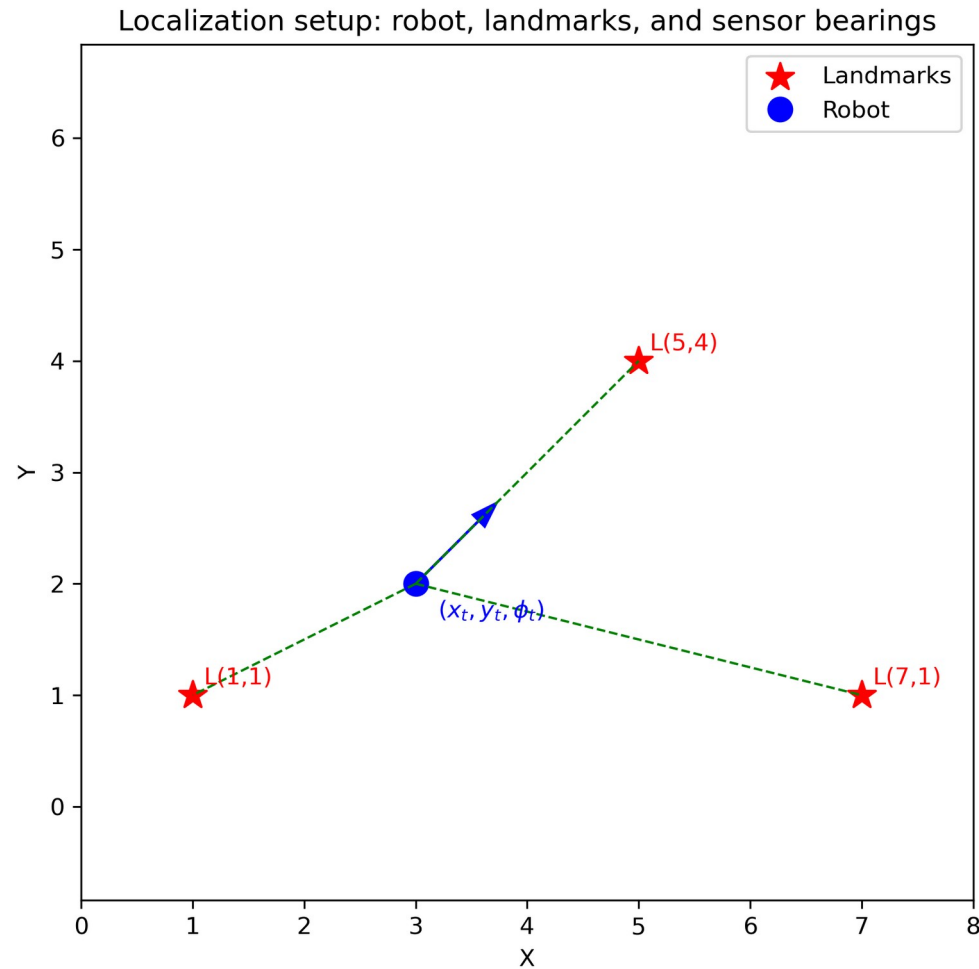
General problem

- Given $x_t = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}$ in 2D or in general $x_t \in SE(2)$ or $SE(3)$
- We use two models
 - A motion (or action) model that describes how the robot's pose evolves given control inputs u_t
$$x_t = f(x_{t-1}, u_t) + \text{noise}$$
 - An observation (or sensor) model that relates the robot's pose to expected sensor readings z_t
$$z_t = h(x_t, m) + \text{noise}$$

where m is an optional reference (map)
- Localization is the inverse problem of motion and perception
 - Given z_t, u_t we want to infer x_t

Localization

General problem



Localization

Formal problem

- Localization can be formulated in two ways:
 - Deterministically (single best estimate)

$$\hat{x}_t = \arg \min_{x_t} ||z_t - h(x_t, m)||$$

- Probabilistically (belief representation)

$$bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t}, m)$$

Updated recursively using (Bayesian update)

$$bel(x_t) = \eta p(z_t \mid x_t, m) \int p(x_t \mid u_t, x_{t-1}), bel(x_{t-1}), dx_{t-1}$$

Reminder: Bayes theorem

$$p(A|B) = \frac{p(B|A)p(A)}{p(B)}$$

Deterministic Localization

Some deterministic localization methods

- Odometry / Dead-Reckoning
Wheel encoders, IMU and motor commands fusion
- Triangulation / Trilateration
Beacons / GNSS
- Vision-based pose estimation
Homography, P3P, Perspective-n-Point
- Scan matching
ICP, NDT
- Map Matching

Vision-based pose estimation

Perspective-n-Point problem

- The robot observes landmarks in the environment
- Landmarks 3D positions are known in a fixed frame (map)

$$P_i = [X_i, Y_i, Z_i]^T$$

- Extract 2D points from image

$$p_i = [u_i, v_i]^T$$

- Establish correspondence between 2D points and 3D landmarks
- Determine the extrinsic matrix of the camera

$$s_i \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} = K[R|t] \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Vision-based pose estimation

Perspective-n-Point problem

- Determining the extrinsic matrix (pose) requires minimizing the reprojection error:

$$\hat{R}, \hat{t} = \arg \min_{R, t} \sum_{i=1}^n ||p_i - \pi(K[R|t]P_i)||^2$$

- Closed form exists for 3 correspondences (P3P) (4 possible solutions)
- Efficient PnP solves the general problem for 4 or more correspondences (Linear least squares)
- Maximum accuracy (refinement) requires nonlinear optimization (e.g., Gauss-Newton)

Scan matching

Iterative closest point

- Estimate the relative pose between two sets of points
 - Given two sets of points (fixed F and moving M)
 - Associates each point from the moving set p_i with the closest one from the fixed set m_i
 - Estimates the pose that minimizes the mean squared distance between pairs of points
 - Repeat until convergence
- Mathematically:

$$\min_{R,t} E(R,t)$$

with

$$E(R,t) = \frac{1}{N} \sum_{i=1}^N \|\mathbf{m}_i - (\mathbf{R} \cdot \mathbf{p}_i + \mathbf{t})\|^2$$

Scan matching

Iterative closest point

- Closed-form solution per iteration

Compute centroids

$$\bar{\mathbf{p}} = \frac{1}{N} \sum_i \mathbf{p}_i, \quad \bar{\mathbf{m}} = \frac{1}{N} \sum_i \mathbf{m}_i$$

Subtracts centroids

$$\tilde{\mathbf{p}}_i = \mathbf{p}_i - \bar{\mathbf{p}}, \quad \tilde{\mathbf{m}}_i = \mathbf{m}_i - \bar{\mathbf{m}}$$

Compute cross-variance and perform SVD

$$\mathbf{H} = \sum_i \tilde{\mathbf{p}}_i \cdot \tilde{\mathbf{m}}_i = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

Compute rotation and translation

$$\mathbf{R} = \mathbf{V} \mathbf{U}^T, \quad \mathbf{t} = \bar{\mathbf{m}} - \mathbf{R} \bar{\mathbf{p}}$$

- Full algorithm
 - Transform moving points using current estimate
$$\mathbf{p}_i' = \mathbf{R} \mathbf{p}_i + \mathbf{t}$$
 - Find correspondences between $\mathbf{p}_i'$ and $\mathbf{m}_i \in M$
 - Compute optimal pose using closed form solution
 - Update pose
$$\mathbf{R} \leftarrow \mathbf{R}_{\text{new}} \mathbf{R}, \mathbf{t} \leftarrow \mathbf{R}_{\text{new}} \mathbf{t} + \mathbf{t}_{\text{new}}$$
 - Check convergence
 - Error below a threshold
 - Maximum iterations reached

Scan matching

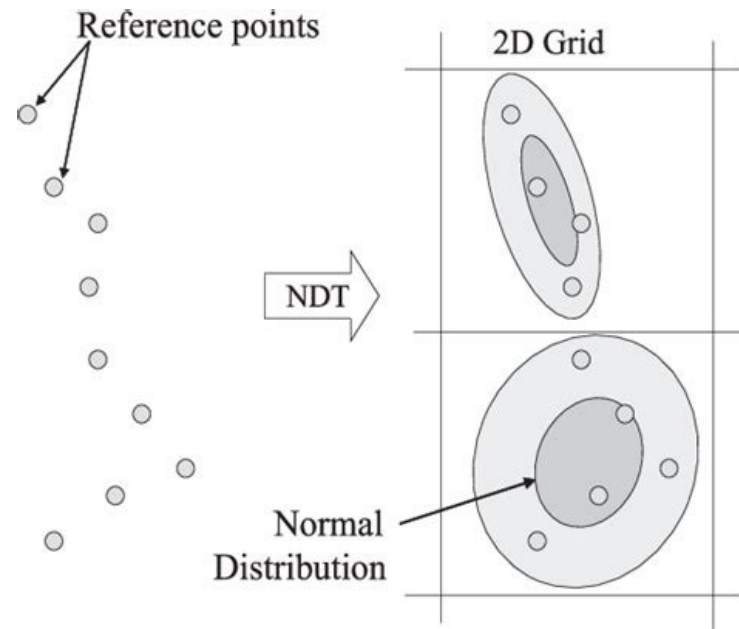
Iterative closest point

- Strengths
 - Simple
 - Works well in practice for small inter-scan motion
 - Efficient correspondence using spatial data structure (KD-tree nearest-neighbor search)
 - No explicit feature extraction
- Limitations
 - Requires a good initial guess
 - May converge to local minima
 - Sensitive to outliers and partial overlaps
 - Expensive for large point sets

Scan matching

Normal distribution transform

- Probabilistic deterministic scan matching
- Reference scan is represented as a set of Gaussian distributions
- Matching the moving scan to the fixed one is equivalent to maximizing the likelihood
- Optimization is smooth and differentiable (enables gradient-based optimization)



Scan matching

Normal distribution transform

- Reference scan transformation
 - Divide the space into grid cells
 - Compute mean and covariance matrix for each non-empty cell i

$$\mu_i = \frac{1}{n} \sum_{k=1}^n \mathbf{m}_k, \quad \Sigma_i = \frac{1}{n} \sum_{k=1}^n (\mathbf{m}_k - \mu_i)(\mathbf{m}_k - \mu_i)^T$$

- Gaussian likelihood for a cell
 - For a point $\mathbf{p}_j$ from the moving set that falls into a grid cell i we compute

$$f_i(\mathbf{p}_j) = \frac{1}{2\pi\sqrt{\det \Sigma_i}} e^{-\frac{1}{2}(\mathbf{p}_j - \mu_i)^T \Sigma_i^{-1} (\mathbf{p}_j - \mu_i)}$$

- Objective function to maximize is the sum of log-likelihood

$$\mathcal{L}(\mathbf{R}, \mathbf{t}) = \sum_{j=1}^N \log(f_i(\mathbf{p}_j))$$

Scan matching

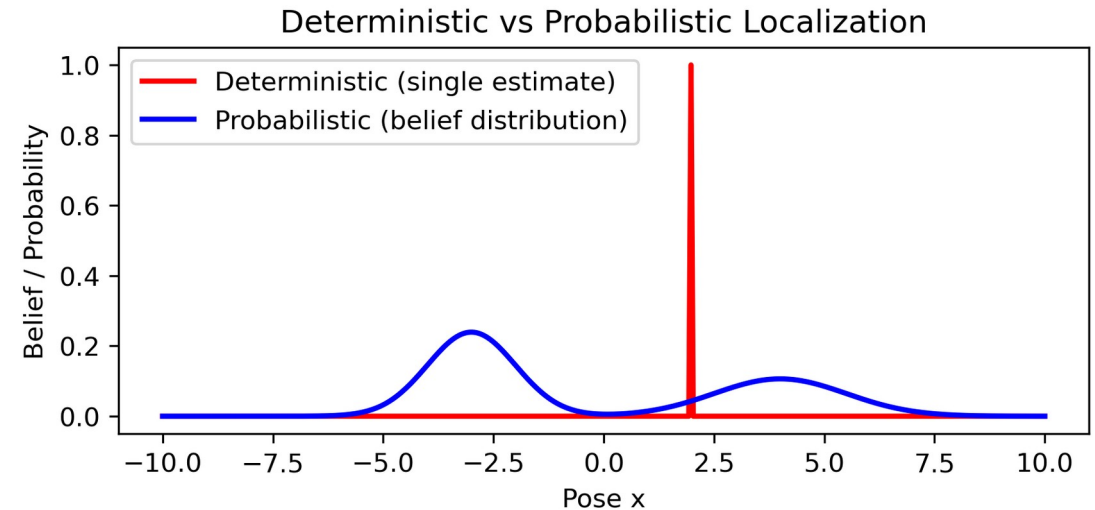
Normal distribution transform

- Full algorithm
 - Build Gaussian transform grid from fixed scan
 - Initialize optimization with coarse estimate
 - Iteratively:
 - Transform moving scan using current estimate
 - Find the grid cell that contains each transformed point
 - Compute the Jacobian (possibly Hessian) of the objective function
 - Perform single step of Gauss-Newton (or Levenberg-Marquart)
 - Check convergence
 - Error below a threshold
 - Maximum iterations reached

Probabilistic localization

Motivation

- World is uncertain
 - Sensor noise
 - Reference / map error
 - Dynamic environment
- Deterministic models may fail if there is a mismatch
 - Robot may become lost
- We need a model robust to noise / partial matches
=> Markov localization framework



Probabilistic localization

Markov localization

- Represent the belief of the robot as a probability distribution over all possible states

$p(x_t)$ = probability that the robot is at state x_t at time t

- The distribution may not be unimodal => we can track multiple hypothesis
- Belief can be updated using recursive Bayesian update

- Prediction step (prior computation)

- When the robot moved according to a control command u_t

$$\bar{p}(x_t) = \int p(x_t | u_t, x_{t-1}) p(x_{t-1}) dx_{t-1}$$

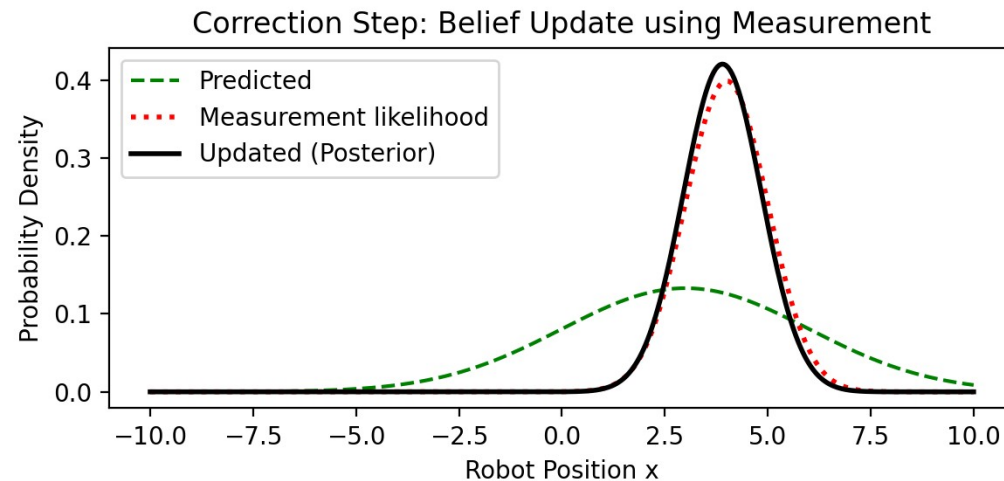
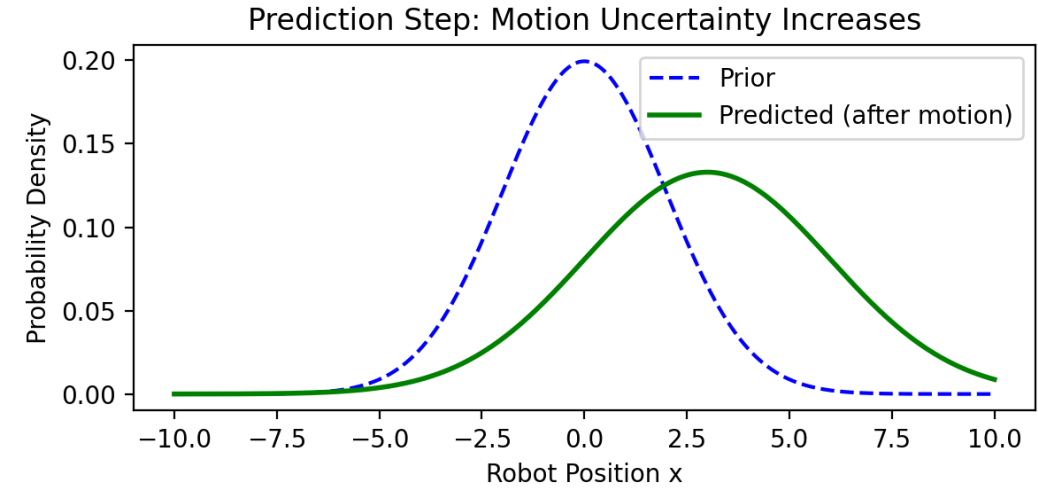
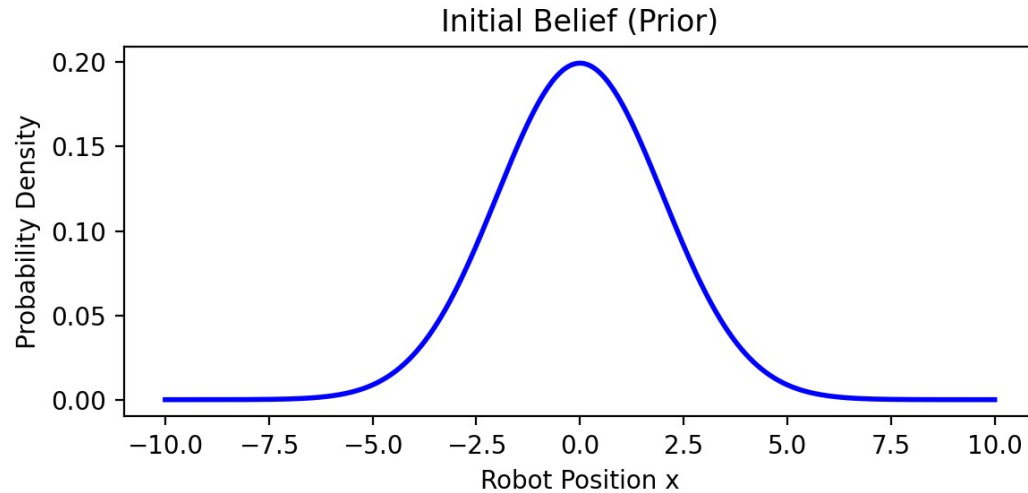
- Correction step (posterior computation)

- When the robot received sensor measurement z_t

$$p(x_t) = \eta p(z_t | x_t) \bar{p}(x_t)$$

Probabilistic localization

Markov localization



Probabilistic localization

General sensor model

- Given a pose x_t , an observation z_t and a reference (e.g., map) m we define the evidence function

$$p(z_t|x_t, m) = \prod_k p(z_t^k|x_t, m)$$

- In practice we often use the log-likelihood instead
 - Faster to compute
 - Optimization friendly
 - No underflow

$$\log p(z_t|x_t, m) = \sum_k \log p(z_t^k|x_t, m)$$

Probabilistic localization

Probability density function

- A probability density function (PDF) $p(x)$ describes how probability is distributed over a continuous variable x
- The probability of x being in an interval $[a,b]$ is:

$$P(a \leq x \leq b) = \int_a^b p(x) dx$$

- Most common model for sensor noise is the normal PDF

$$\mathcal{N}(x; \mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right)$$

Probabilistic localization

Range sensor model

- Given a robot at pose $x=(p_x, p_y, \theta)$ in a map m
- Raycast into the map to obtain the expected ranges z_k^*
- The range sensor model is a mixture

$$p(z_k|x, m) = \underbrace{\eta \cdot z_{\text{hit}} \cdot \mathcal{N}(z_k; z_k^*, \sigma_{\text{hit}}^2)}_{\text{expected}} + \underbrace{z_{\text{short}} \cdot \exp(-\lambda z_k) \cdot \mathbb{I}(z_k < z_k^*)}_{\text{obstacle}} + \underbrace{z_{\text{max}} \cdot \mathbb{I}(z_k = z_{\text{max}})}_{\text{failure}} + \underbrace{z_{\text{rand}} \cdot U[0, z_{\text{max}}]}_{\text{random}}$$

- z_{hit} , z_{short} , z_{max} and z_{rand} are weights (sum to 1)

Probabilistic localization

Camera sensing model

- Suppose the robot sees a landmark at pixel coordinates corresponding to an angle θ_z from the camera
- If the landmark is at map location $l = (x_l, y_l)$, the expected bearing from robot pose $x_t = (x, y, \phi)$ to the landmark is

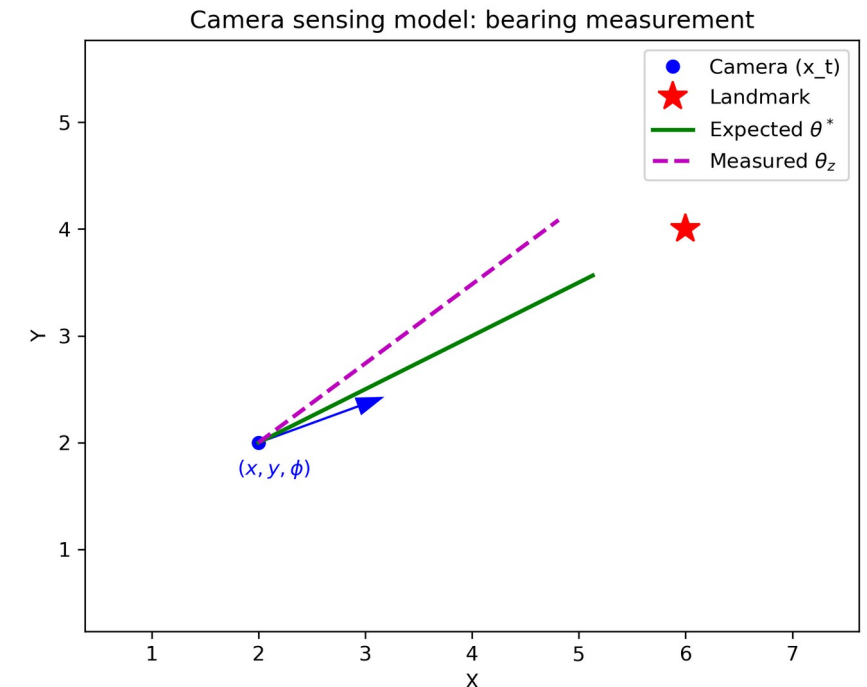
$$\theta^* = \tan^{-1}\left(\frac{y_l - y}{x_l - x}\right) - \phi$$

- The measurement likelihood (assuming Gaussian noise) is

$$p(z_t | x_t, m) = \mathcal{N}(\theta_z; \theta^*, \sigma_\theta^2)$$

- If depth r is also available, combine with bearing

$$p(z_t | x_t, m) = \mathcal{N}(r; r^*, \sigma_r^2) \cdot \mathcal{N}(\theta_z; \theta^*, \sigma_\theta^2)$$



Probabilistic localization

Markov localization in practice

- State space is infinite
- Computers cannot process infinite state spaces in real-time
- $\Rightarrow$ We work in discrete state space
- Various implementations
 - Kalman Filter (single hypothesis)
 - Grid-based (occupancy) localization
 - Particle filter / Monte Carlo localization

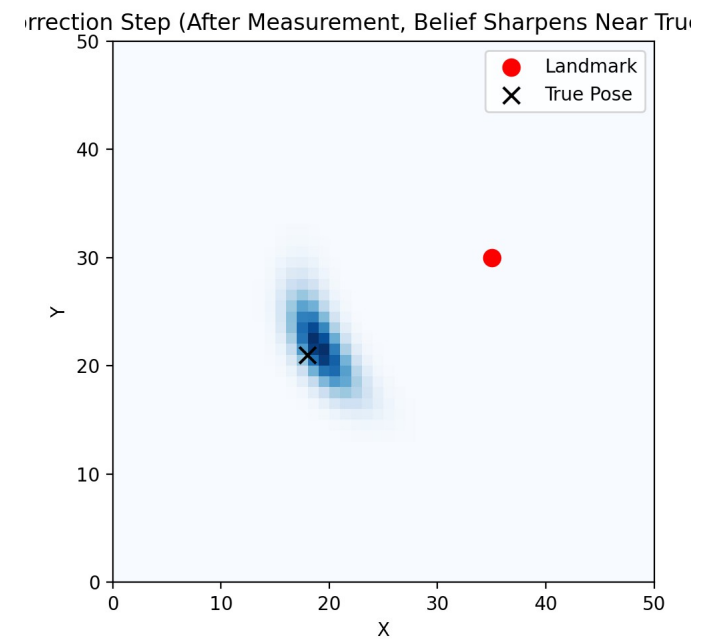
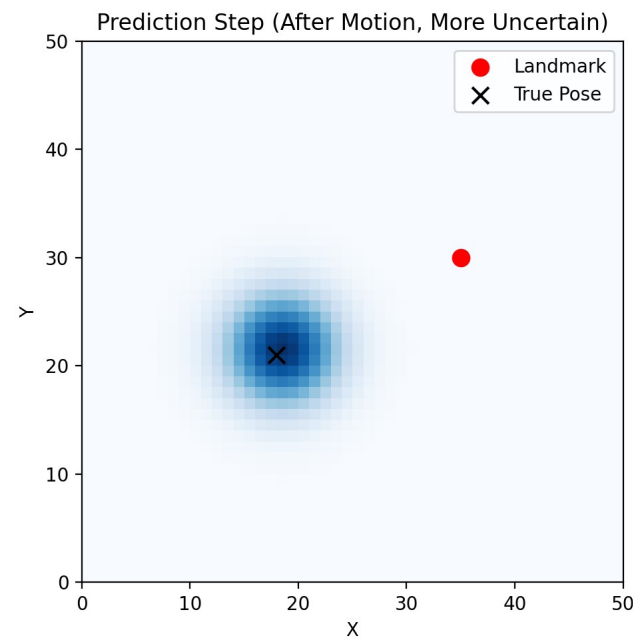
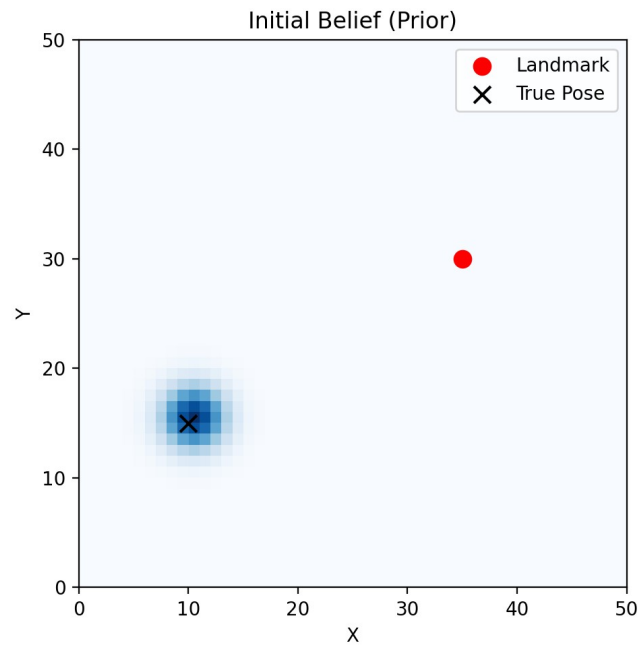
Probabilistic localization

Grid-based localization

- The belief is discretized over a grid
- For each grid cell, we store the probability of the robot being at that location
- Recursive update of belief:
 - Prediction step: Update cell probabilities based on robot's motion
 - Correction step: Update cell probabilities based on sensor measurements
- Use the Maximum a Posteriori (MAP) estimate or the expected value as current pose
- Advantages:
 - Simple to understand and implement
 - Works well in environments with well-defined landmarks (e.g., maps)

Probabilistic localization

Grid-based localization



Probabilistic localization

Particle filter

- Particle filter (or Monte Carlo localization) approximates a continuous belief distribution using a set of particles
 - Each particle represents a hypothesis about the robot's state
 - Recursive update of belief:
 - Prediction step: Move particles based on the robot's motion model
 - Correction step: Weight particles according to their likelihood based on sensor readings
- Use the Maximum a Posteriori (MAP) estimate or the expected value as current pose
- Advantages:
 - Works in large, continuous state spaces
 - Flexible and can handle non-Gaussian distributions and multi-modal beliefs
 - More scalable than grid-based methods in environments with high-dimensional state spaces

Probabilistic localization

Particle filter

